

实验项目名称: 实验三-文本单模态深度学习

实验项目名称：实验三-文本单模态深度学习

一.实验背景与目标

随着人工智能技术的不断发展，情感识别已经成为人机交互、智能语音助手、情感计算等领域的重要研究方向。情感识别旨在通过分析人类的语音、表情、文本等输入信号，识别并理解其情感状态。在本实验中，更加专注于文本单模态情感识别，目标是利用深度学习方法通过使用预训练bert模型文本特征的自动提取与分析，建立一个能够准确识别多种情感状态的模型。具体来说，本实验希望通过对话中的文本特征，准确识别四种情感状态：愤怒、快乐/兴奋、中性和悲伤。

实验目标包括：

- 通过bert模型提取文本token，计算文本特征，从而获取情感相关特征信息；
- 使用深度学习算法训练情感分类模型；
- 评估所构建模型的准确性与鲁棒性，并且得到最终的test_result文件。
- 我总共提交了两个实验python文件，new.py代表最佳模型，chaocan.py表示搜索过程

二.实验方法

本实验采用多种深度学习训练技巧方法，结合文本特征处理技术以及数据预处理，通过提取对话中的文本特征进行情感分类，通过提取文本信息的特征，训练模型以识别四种情感状态：愤怒、快乐/兴奋、中性、悲伤，而提高最终模型性能的关键在于数据预处理、特征提取和最终模型训练。

具体的实验流程包括以下几个主要步骤：

- **数据预处理**：对原始对话文本数据进行清洗，包括去除无意义的符号、停用词过滤、词干提取等操作，以提高数据质量。此外，还将添加上下文信息、带有情感标签的句子，以及通过翻译等方式增加文本内容，以便提取更丰富的特征。
- **特征提取**：使用预训练的bert模型，提取文本的深层次语义特征。bert模型通过对大量文本数据的预训练，能够捕捉到丰富的语言模式和语义信息，这对于情感识别任务至关重要。
- **模型训练**：基于提取的文本特征，采用深度学习算法，连接一个分类头或者textcnn对提取出来的向量特征进行模型训练。利用训练集和验证集进行验证，以优化超参数，提高模型的泛化能力。
- **性能评估**：通过准确率、精确率、召回率、F1分数等常用评价指标对模型在测试集上的性能进行评估，确保模型的准确性与鲁棒性。

下面进行详细介绍，其中特征提取和模型训练有重叠，不分开讲解：

数据预处理

数据预处理是文本特征处理中至关重要的一步，它的主要目的是清洗和丰富对话文本内容，让模型能够更好地理解上下文内容，从而让模型懂得情感的发展逻辑，以便后续的特征提取与模型训练。针对本实验的对话文本数据集，预处理过程包括以下几个关键步骤：

- **文本清洗**：去除文本中的无意义符号、停用词过滤、词干提取等操作，以提高数据质量。这一步骤有助于减少噪声并提取出对情感识别更有帮助的特征。在实际数据集上，我发现文本中有 [laughter] 这种旁白信息，这些信息很明显对情感分析的作用巨大，因此我尝试将让模型理解 [laughter] 很重要，因此尝试将文本内容重复三遍以及 <<<laughter>>> 这样表示信息，有一点作用，但是提升不大。结合 [cls] 的表示，可能本身模型就对 [] 中的内容很重视，因此最终没有修改该部分。

```
def annotate_emotion_marks(text):  
    # return re.sub(r"\[(.*?)\]", lambda match: f"[{match.group(1)}]  
    [{match.group(1)}][{match.group(1)}]", text)  
#gai 中性能有所提升
```

- **上下文信息添加**：在对话中，上下文对于理解情感状态非常重要。因此，我将对话中的**当前说话人的信息以及该句话长度整合到单个样本中**，以便模型能够捕捉到情感的变化和发展。想到这个方法的原因是我想到男女在进行情感表达时有较大的差别，如果能把这个信息充分利用，必然能让情感分析更加细腻与易预测，比如女生的情感相比于男生更难预测，那么模型在对男生预测时就更加直白，女生则可能需要考虑更多内容，后续观察到数据集中的id代表了谈话的性别、主题、谈话人、第几段，这些内容如果模型能够理解，那么我在数据阶段就充分实现了模型上下文关系的理解，相对于修改模型更加方便和直观。后续又想到是否可以附加一些其他内容来帮助模型理解对话人的情感，自然地想到了可以添加文本的长度，虽然bert模型对数值信号变化不敏感，但是应该可以根据长度来作为阈值判断情感，比如人在极度悲伤和愤怒时说的话可能就少一点，开心或激动情况下话会变多。

```
4#Ses01F_impro01_F005#train/Ses01F_impro01_F005.wav#well what's the problem?  
Let me change it.#2
```

因此最后选择在开头加入 id: 代表该说话人将要表达后续内容，在结尾加入句子长度信息，虽然这个方法看起来简单，但是其实对效果提升帮助很大，我刚开始尝试在全部训练样本上跑的时候f1差不多为0.68，使用该方法针对文本进行预处理后，性能直接飙升到0.76，如果我比其他人模型性能好，核心点就在这个地方。

- **情感标签添加**：为了使模型能够学习到情感状态，我将带有情感标签的句子与对应的文本结合，这样模型在学习过程中可以同时关注文本内容和情感标签。尝试使用ntlk中的情感分析内容，根据阈值判断从而提高标签质量，但是效果不明显，说明数据集本身的标签已经经过很好的处理了。

```
def apply_vader_sentiment(text):  
    # 初始化VADER情感分析器  
    analyzer = SentimentIntensityAnalyzer()  
    sentiment_score = analyzer.polarity_scores(text)  
  
    # 根据 compound 分数判断情感类型  
    if sentiment_score['compound'] > 0.2: # 高于阈值则认为是积极情感  
        return "happy or excited"  
    elif sentiment_score['compound'] < -0.2: # 低于阈值则认为是负向情感  
        return "angry"  
    else: # 否则认为是中立情感  
        return "neutral"
```

- **文本增强**：通过翻译、同义词替换、句子重组等方式增加文本内容，这有助于模型学习到更多样的情感表达方式，提高模型的泛化能力。尝试将对话内容转为法语从而增强数据，有效果的提升，可能只是重复性地表达句子，而没有理解其中的情感。

```
def preprocess_text_with_conversation_id_trans(csv_file):
    # 读取CSV文件
    data = pd.read_csv(csv_file, sep="#")

    # 处理原始文本
    data['enhanced_original_text'] = data['id'] + ": " + data['text']
    data['original_text_length'] = data['text'].apply(len)
    enhanced_original_texts = data['enhanced_original_text'] + " (" +
    data['original_text_length'].astype(str) + ")"

    # 处理翻译文本
    data['enhanced_translated_text'] = data['id'] + ": " + data['translated']
    data['translated_text_length'] = data['translated'].apply(len)
    enhanced_translated_texts = data['enhanced_translated_text'] + " (" +
    data['translated_text_length'].astype(str) + ")"

    # 将原始文本和翻译文本合并到一个总的列表中
    total_texts = list(enhanced_original_texts) + list(enhanced_translated_texts)

    # 标签也需要重复一次，因为每个文本（原始和翻译）都对应同一个标签
    total_labels = list(data['label']) + list(data['label'])
    return total_texts, total_labels
```

最后选择的数据预处理代码如下：

```
def preprocess_text_with_conversation_id(csv_file):
    """
    从CSV中读取数据，将对话名（id字段）加到文本内容前，并计算文本长度。
    :param csv_file: CSV文件路径
    :return: 新的文本列表和标签列表（含长度信息）
    """
    data = pd.read_csv(csv_file, sep="#")
    # data['text'] = data['text'].apply(annotate_emotion_marks)
    # # 使用 VADER 对文本进行情感分析并生成情感标签
    # data['vader_label'] = data['text'].apply(apply_vader_sentiment)
    # 合并对话名和文本
    data['enhanced_text'] = data['id'] + ": " + data['text']
    data['text_length'] = data['text'].apply(len) # 计算文本长度
    # # 如果情感标签和原标签冲突，选择VADER的情感标签（根据需要）
    # data['final_label'] = data['vader_label'] # 使用VADER标签作为最终标签
    # 添加文本长度到结果中
    enhanced_with_length = data['enhanced_text'] + " (" +
    data['text_length'].astype(str) + ")"

    # 返回处理后的文本和标签
    return list(enhanced_with_length), list(data['label'])
```

在本实验中，我重点关注了文本清洗和文本增强两个方面。文本清洗将确保模型接收到的数据尽可能地干净和充分利用，而文本增强则旨在通过增加样本多样性来提高模型的鲁棒性。在比较复杂的情感表达中，特别是验证集（dev）上，这样的预处理可以帮助模型更好地捕捉情感特征，但预处理步骤对于提高模型性能仍然是非常必要的。

特征提取和模型训练

本实验旨在通过对话文本单模态数据实现情感识别，在特征提取和模型训练上，尝试使用了不同的bert模型和不同的参数，并且对模型架构进行微调或者对分类层进行修改，并且使用贝叶斯优化进行参数调整，下面进行详细地介绍我已经做过的工作。

在数据预处理完成后，特征提取和模型训练是实现文本单模态情感识别的关键步骤。以下是模型训练的具体流程：

1. 模型选择

对于文本情感识别任务，我们选择基于深度学习的bert模型，开始使用实验本身提供的带有初始参数的bert，后续经过搜索和尝试，使用了预训练的RoBERTa模型。RoBERTa（Robustly optimized BERT approach）是一种基于Transformer架构的预训练语言模型，它在大规模的文本数据上进行预训练，能够捕捉到**丰富的语言模式和语义信息**，非常适合用于情感分析等自然语言处理任务。

2. 模型架构

使用RoBERTa模型作为基础架构，通过微调的方式适应我们的情感识别任务。RoBERTa模型包含多层Transformer编码器，能够处理输入的文本序列，并输出每个token的嵌入表示。这些嵌入表示将被送入后续的全连接层，用于情感分类。同时，我也尝试使用将嵌入表示显示输出，将其送入到textcnn进行分类训练。

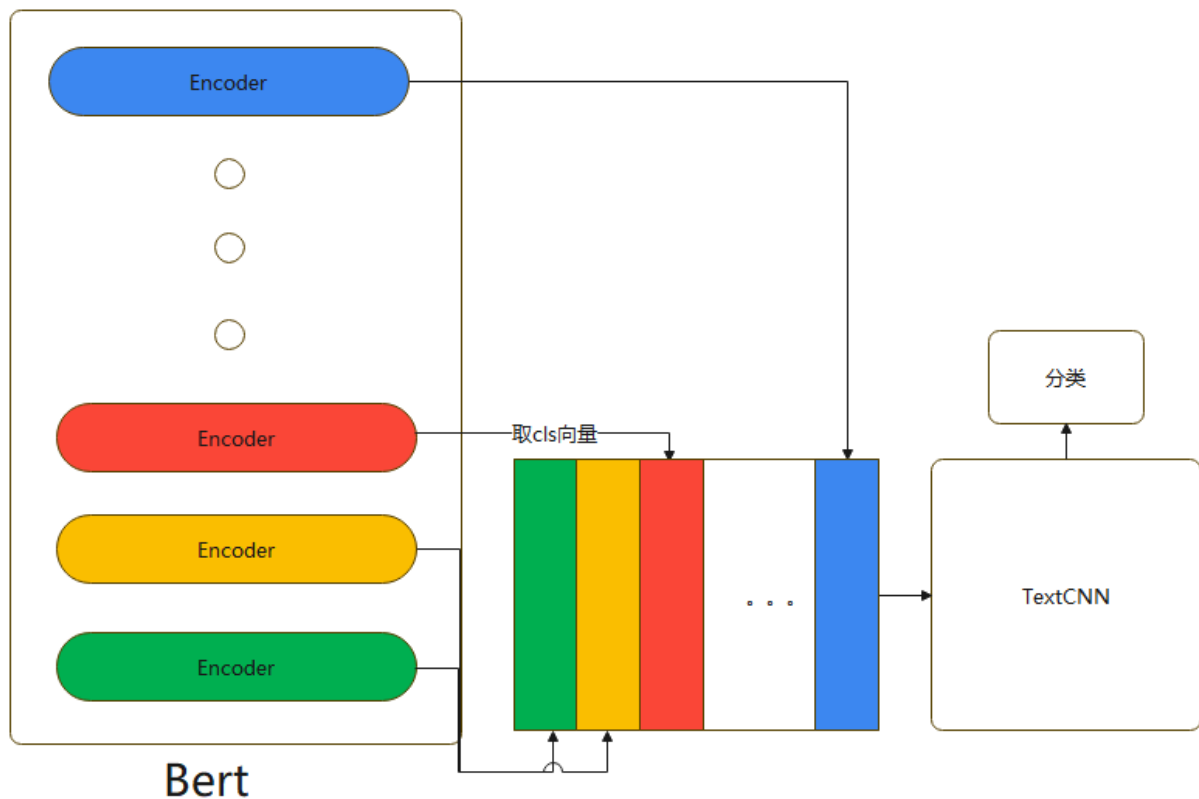
第一种方法是将嵌入表示显示输出，将其送入到textcnn进行分类训练，能针对最终结果进行涨点，但是不稳定、容易过拟合，所以最终没有采用该模型架构。

```
class TextCNN(nn.Module):
    def __init__(self, embedding_dim, num_filters, filter_sizes, output_dim,
                 dropout=0.5):
        super(TextCNN, self).__init__()
        self.convs = nn.ModuleList([
            nn.Conv2d(1, num_filters, (fs, embedding_dim)) for fs in filter_sizes
        ])
        self.bns = nn.ModuleList([nn.BatchNorm1d(num_filters) for _ in
                                   filter_sizes])
        self.fc = nn.Linear(len(filter_sizes) * num_filters, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = x.unsqueeze(1) # Add channel dimension (N, 1, seq_len,
                           embedding_dim)
        convd = [conv(x) for conv in self.convs] # (N, C, H, 1)
        convd = [conv.squeeze(3) for conv in convd] # (N, C, H)
        convd = [self.bns[i](conv) for i, conv in enumerate(convd)] # Apply BN
        convd = [F.relu(conv) for conv in convd] # Apply ReLU
        pooled = [F.max_pool1d(conv, conv.shape[2]).squeeze(2) for conv in
                  convd] # (N, C)
        pooled = [self.dropout(p) for p in pooled] # Apply dropout
```

```
cat = torch.cat(pooled, dim=1) # (N, C * len(filter_sizes))
return self.fc(cat)
```

第二种方法是将encoder每一层的输出拼接起来，投入到textcnn中进行训练



Bert-Base除去第一层输入层，有12个encoder层，每个encode层的第一个token (CLS) 向量都可以当作句子向量，我们可以抽象的理解为，encode层越浅，句子向量越能代表低级别语义信息，越深，代表更高级别语义信息。我们的目的是既想得到有关词的特征，又想得到语义特征，模型具体做法是将第1层到第12层的CLS向量，作为CNN的输入，分类，但是评估该方法效果不好。

```
class TextCNN(nn.Module):
    def __init__(self, input_size, num_filters, filter_sizes, output_dim,
                 dropout=0.5):
        super(TextCNN, self).__init__()
        self.convs = nn.ModuleList([
            nn.Conv1d(input_size, num_filters, kernel_size=fs) for fs in
            filter_sizes
        ])
        self.bns = nn.ModuleList([nn.BatchNorm1d(num_filters) for _ in
            filter_sizes])
        self.fc = nn.Linear(len(filter_sizes) * num_filters, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # x shape: (N, 12, 768)
        x = x.permute(0, 2, 1) # (N, 768, 12)
        convd = [conv(x) for conv in self.convs] # (N, C, H)
        convd = [F.relu(conv) for conv in convd] # Apply ReLU
        pooled = [F.max_pool1d(conv, conv.shape[2]).squeeze(2) for conv in
            convd] # (N, C)
        pooled = [self.dropout(p) for p in pooled] # Apply dropout
```

```

        cat = torch.cat(pooled, dim=1) # (N, C * len(filter_sizes))
        return self.fc(cat)

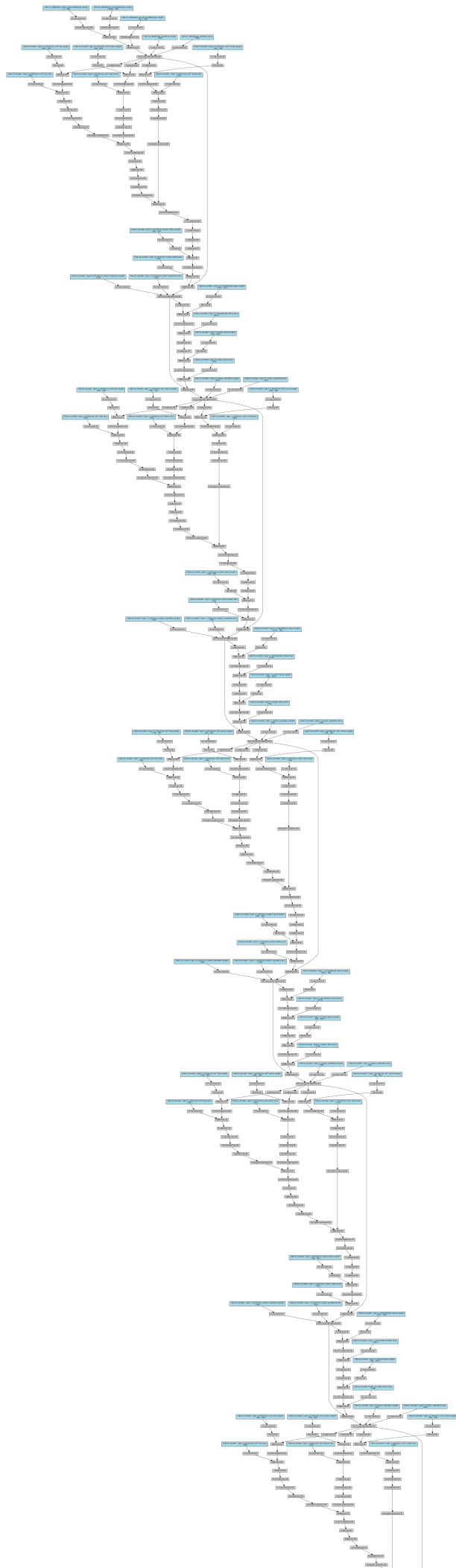
class MyDLmodel:
    def __init__(self, model, device, weight_decay=0.01, dropout_prob=0.1,
num_training_steps=None):
        self.model = model
        self.textcnn = TextCNN(
            input_size=768, # 每个CLS向量的维度
            num_filters=100, # 每个滤波器的数量
            filter_sizes=[2, 3, 4], # 滤波器大小
            output_dim=4, # 分类数
            dropout=dropout_prob
        ).to(device)

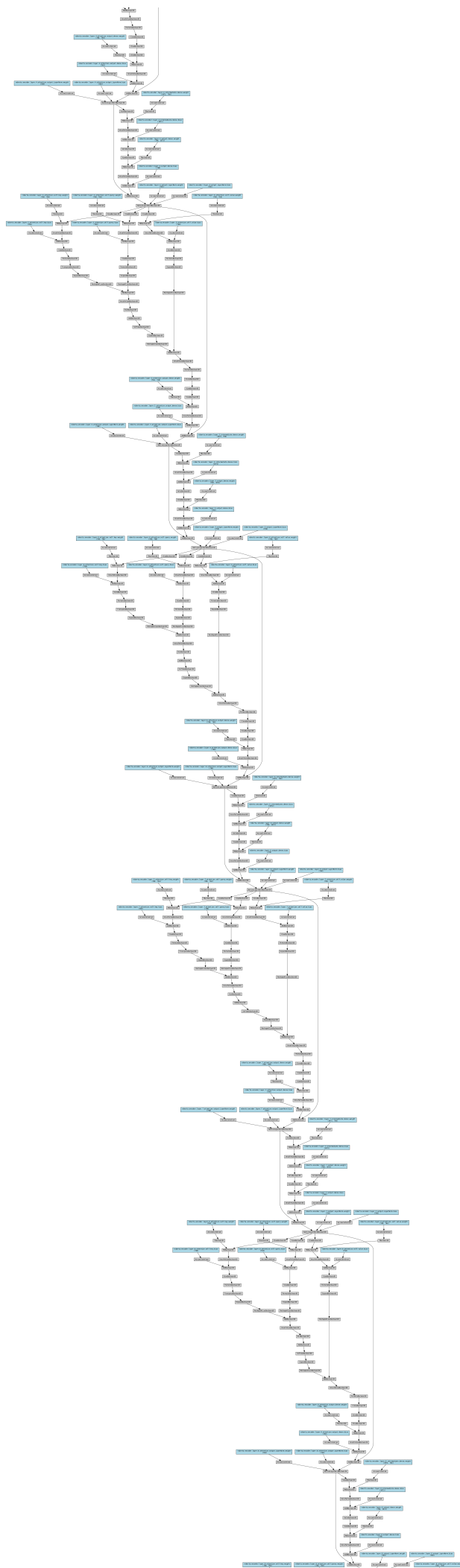
        self.model.to(device)
        self.device = device
        self.optimizer = torch.optim.AdamW(list(self.model.parameters()) +
list(self.textcnn.parameters()), lr=5e-5, weight_decay=weight_decay)
        self.scheduler = get_cosine_schedule_with_warmup(self.optimizer,
num_warmup_steps=0, num_training_steps=num_training_steps)

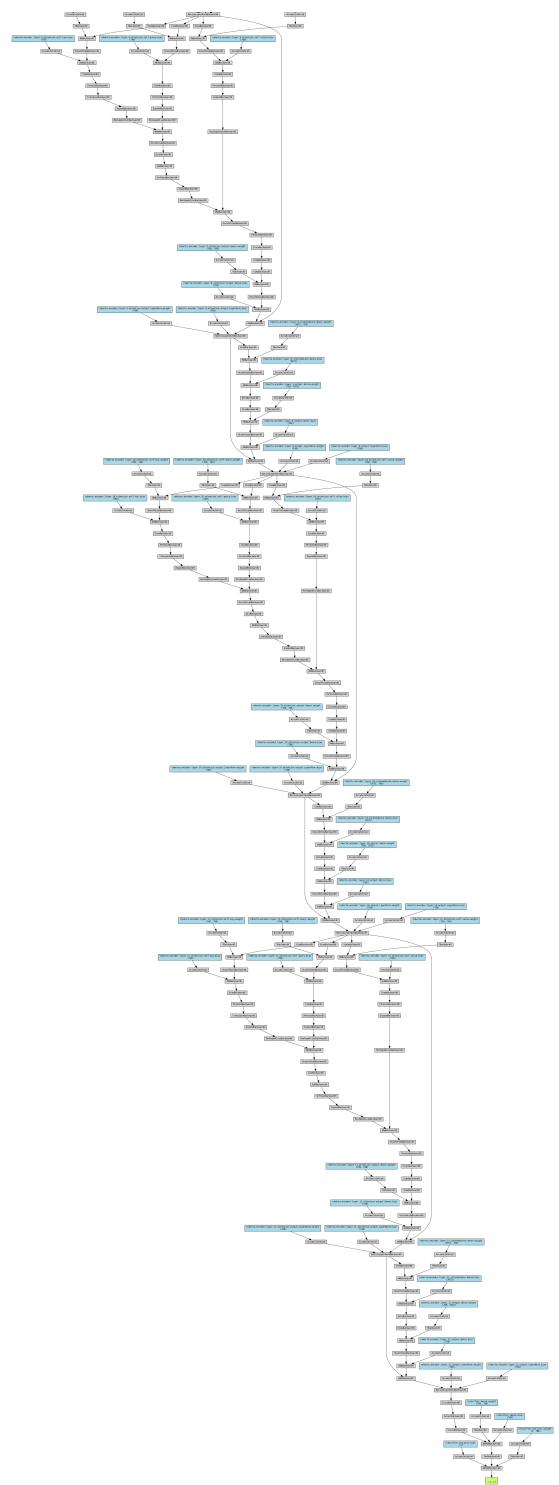
        class_counts = np.array([606, 891, 1066, 696])
        N = class_counts.sum()
        class_weights = N / class_counts
        class_weights[0] *= 1.4
        class_weights[2] *= 1.4
        weights_tensor = torch.tensor(class_weights,
dtype=torch.float32).to(device)
        self.criterion = FocalLoss(alpha=1, gamma=5, weights=weights_tensor)

```

第三种方法就是直接将嵌入表示将被送入后续的全连接层。







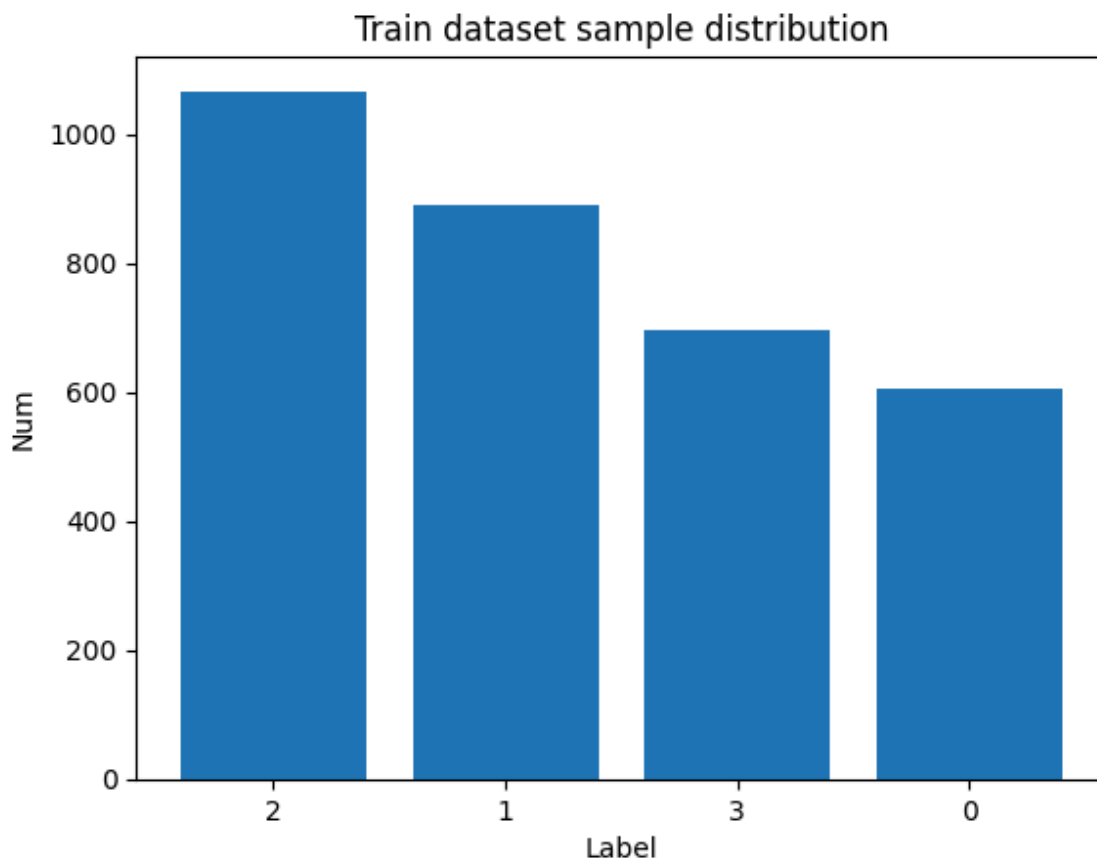
3. 输入数据准备

在模型训练前，我们需要将预处理后的文本数据转换为模型可接受的输入格式。这包括将文本分割为 token，然后使用RoBERTa的tokenizer将token转换为模型可以理解的ID序列(老师给的以及网上下载的bert模型都已经给出了token的vocab列表)。同时，还需要将标签进行编码，以便模型可以进行监督学习。

label	含义
0	angry
1	happy or excited
2	neutral
3	sad

4. 训练设置

- **损失函数**：这是一个四分类问题，我使用交叉熵损失函数来计算预测和真实标签之间的差异。但是样本不平衡会带来模型的性能不好。模型训练的本质是最小化损失函数，当某个类别的样本数量非常庞大，损失函数的值大部分被样本数量较大的类别所影响，导致的结果就是模型分类会倾向于样本量较大的类别。拿当下的标签来说明，会更偏向于第2类，我们的目的是找到让模型能够正确的区分正例和负例，因此，针对当下存在的样本不平衡问题，采用其他的loss方法。



Balanced Loss是解决分类问题中样本类别不平衡的一个方法，通过对样本均衡处理，从而减小样本的分布差异

```
class BalancedLoss(torch.nn.Module):  
    def __init__(self, weights):  
        super(BalancedLoss, self).__init__()  
        self.weights = weights  
  
    def forward(self, logits, labels):  
        loss = F.cross_entropy(logits, labels, weight=self.weights)  
        return loss
```

Focal Loss 就是一个解决分类问题中类别不平衡、分类难度差异的一个 loss

$$L_{fl} = \begin{cases} -\alpha(1-\hat{y})^\gamma \log \hat{y}, & \text{当 } y = 1 \\ -(1-\alpha)\hat{y}^\gamma \log(1-\hat{y}), & \text{当 } y = 0 \end{cases}$$

```

class FocalLoss(torch.nn.Module):
    def __init__(self, alpha=1, gamma=2, weights=None):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.weights = weights

    def forward(self, logits, labels):
        ce_loss = F.cross_entropy(logits, labels, weight=self.weights,
reduction='none')
        p_t = torch.exp(-ce_loss) # For each sample, p_t is the predicted
probability for the true class
        focal_loss = self.alpha * (1 - p_t) ** self.gamma * ce_loss
        return focal_loss.mean() # Average over all batches

```

- **优化器**：我们选择AdamW优化器进行模型参数的更新，它结合了RMSProp和Momentum两种优化算法的优点，适用于大规模数据和参数的场景。
- **学习率**：我们将使用学习率调度器 `get_cosine_schedule_with_warmup` 来动态调整训练过程中的学习率，以避免训练初期的快速收敛和后期的缓慢收敛问题。并且设置预热，使得初始步数缓慢增加，从而避免初期参数变化过快导致学习效果不好。
- **批处理**：为了有效利用计算资源，我们将数据分批次输入模型进行训练，每个批次包含一定数量的样本，这里的batch_size取得16或32，选择较大的batch_size有助于提高模型训练效率。
- **正则化**：为了防止过拟合，我尝试将在模型中加入Dropout层，并使用权重衰减（Weight Decay）作为L2正则化。

5. 模型训练

在训练过程中，按照以下步骤进行：

- **前向传播**：输入数据通过RoBERTa模型，计算得到每个类别的预测概率。
- **计算损失**：使用focalloss损失函数计算预测概率和真实标签之间的损失。
- **反向传播**：根据损失函数的结果，通过反向传播算法更新模型的权重。
- **迭代优化**：重复前向传播和反向传播的过程，直到模型在验证集上的性能不再提升或达到预设的迭代次数。

在训练过程中，我还尝试使用了加入对抗样本来提高模型的泛化性能，但是效果并不理想，可能是对抗样本设置不佳，这是一个很好的思路，但是数据集本身不够大，所以过多的对抗样本反而会降低模型的性能。

6. 超参数调整

我使用网格搜索和贝叶斯优化搜索来寻找最优的超参数组合，以提高模型的性能。

```

lr = trial.suggest_loguniform('lr', 1e-6, 1e-4) # 学习率
alpha = trial.suggest_uniform('alpha', 0.5, 2.0) # Focal Loss 的 alpha
gamma = trial.suggest_uniform('gamma', 0.5, 3.0) # Focal Loss 的 gamma
weight_decay = trial.suggest_loguniform('weight_decay', 1e-6, 1e-2) # 权重衰
减
warmup_fraction = trial.suggest_uniform('warmup_fraction', 0.05, 0.15) #
warmup 步数比例

```

7. 模型验证

在每个epoch结束后，在验证集上评估模型的性能，包括准确率、精确率、召回率和F1分数等指标，并且使用 `tensorboard` 进行记录。有助于监控模型是否过拟合或欠拟合，并根据需要调整训练策略和超参数。

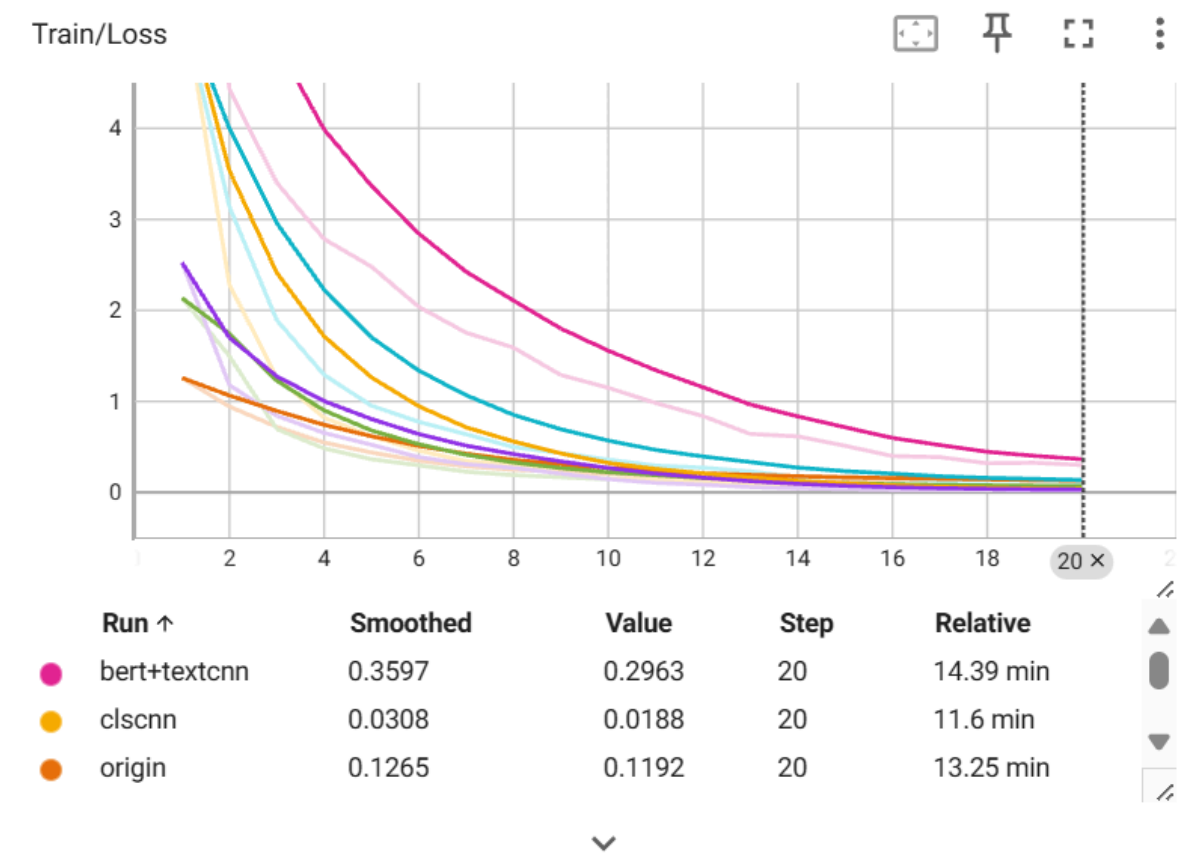
8. 模型保存

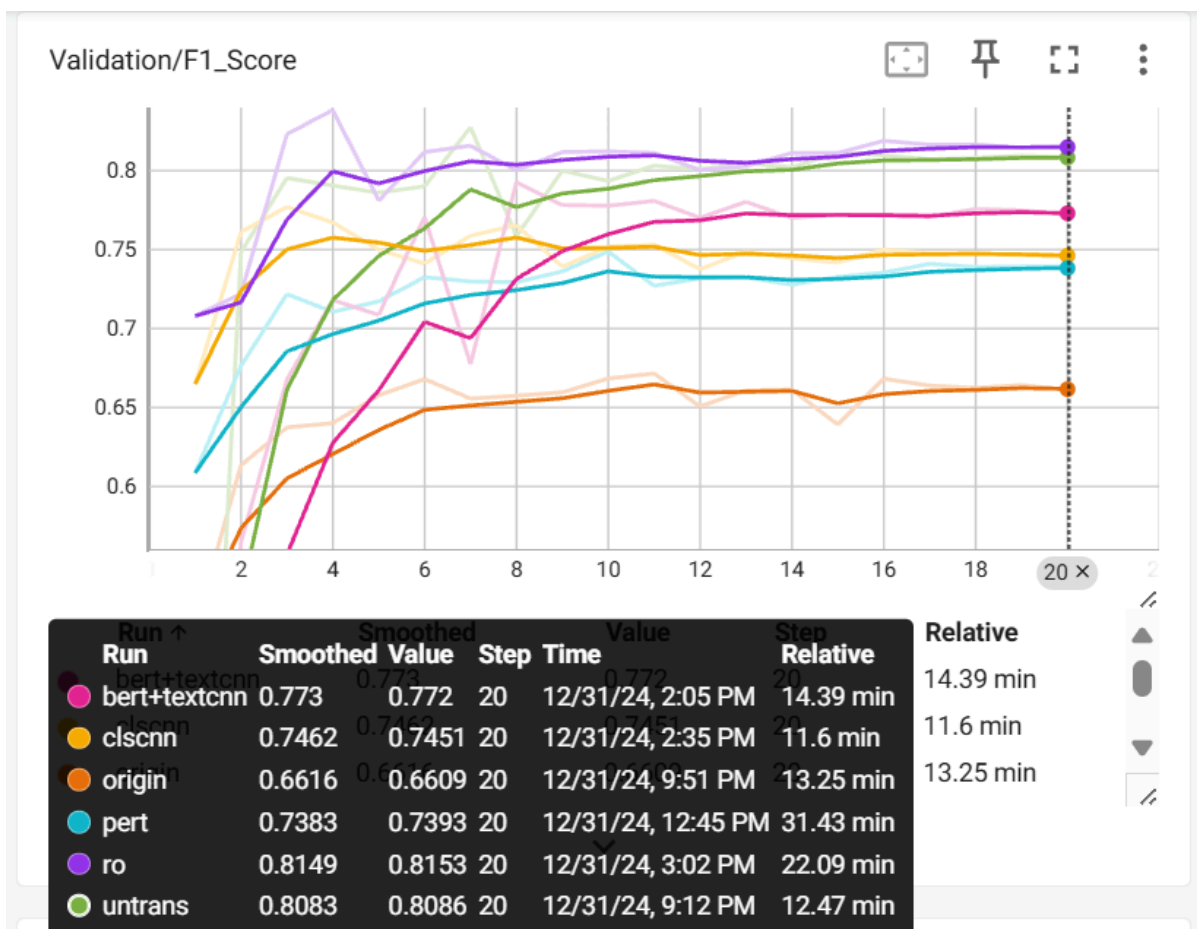
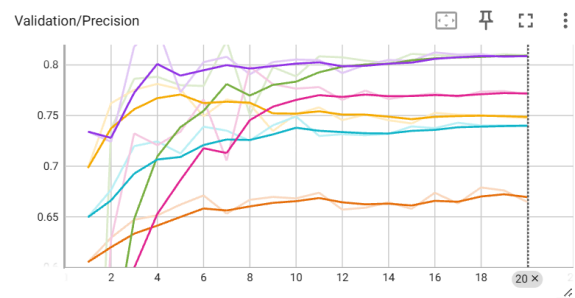
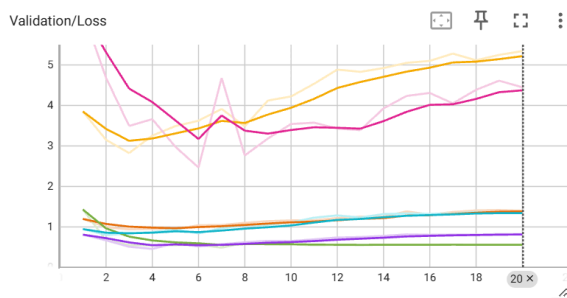
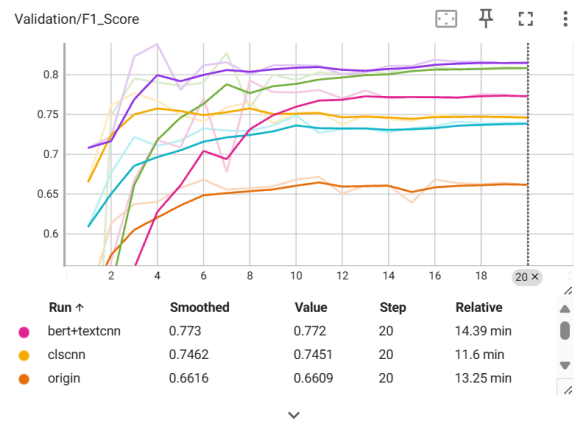
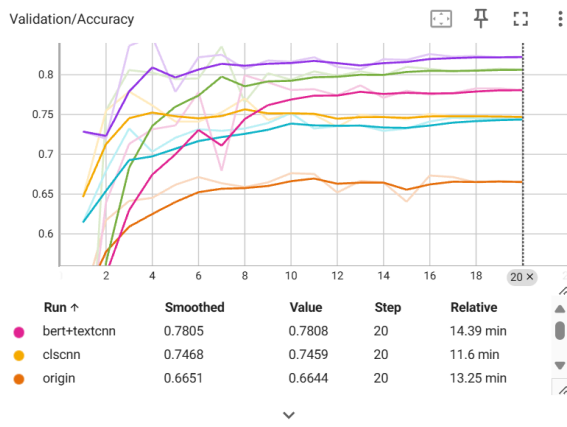
一旦模型在验证集上达到满意的性能，保存模型的权重，以便后续在测试集上进行评估和实际应用。

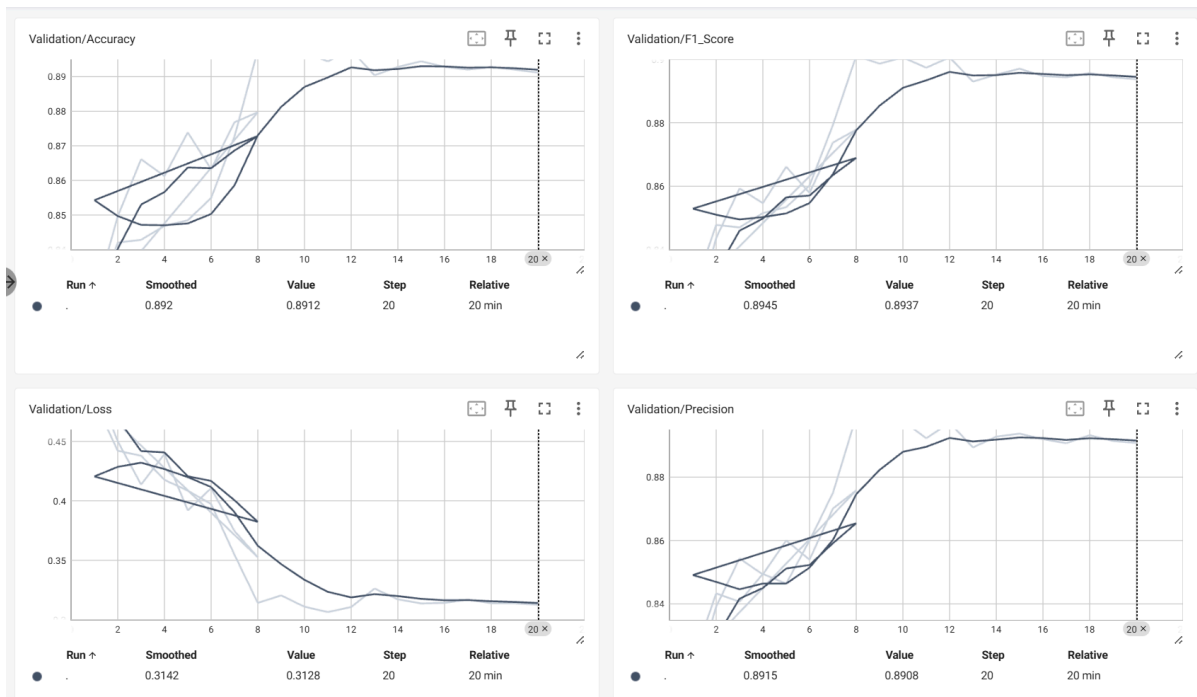
性能评估

使用 `accuracy`、`recall`、`f1`、`precision`、`Confuse_matrix` 来评估模型的性能，并且以 `f1` 为最终评价模型好坏的重要因素，所有的实验数据取得近似最优化的结果展示（都用的是我跑的最优值，可能不是最稳定的值进行的）。

	bert+未数据预处理	bert+加入对抗样本	bert+clscnn	roberta+textcnn	roberta微调+贝叶斯优化	roberta微调+翻译+贝叶斯优化	预训练bert+翻译+贝叶斯优化
accuracy	0.6751	0.7517	0.7701	0.7992	0.8356	0.8477	0.9134
f1	0.6644	0.7486	0.7576	0.7736	0.8275	0.8385	0.9103
precision	0.6721	0.7490	0.7627	0.7721	0.8238	0.8374	0.9094







origin:bert+未数据预处理

pert:bert+加入对抗样本

clsnn:bert+clscnn

bert+textcnn:roberta+textcnn

untrans:roberta微调+贝叶斯优化

ro:roberta微调+翻译+贝叶斯优化

后续实验过程中，发现了较好的预训练模型，采用最新的预训练模型，未经过贝叶斯优化调参就使得模型的性能得到进一步的提高。

最后的：预训练bert+翻译的平均性能在0.89左右，最佳性能f1可以近似达到0.91，是我实现模型的sota

三.实验数据与设置

实验配置

使用了单块**NVIDIA GeForce RTX 3080**（10 GB 版本）

torch==1.10.1+cu111

torchaudio==0.10.1+cu111

torchvision==0.11.2+cu111

实验框架

- 限定使用框架：
 - 机器学习scikit-learn
 - 深度学习pytorch
- 推荐使用版本：
 - python 3.8
 - torch 2.0.1

- torchaudio 2.0.2
- torchvision 0.15.2

数据集说明

总数量：5531

train	dev	test
3259	1031	1241

CSV文件说明

使用“#”分隔

num	id	path	text	label
文件数量序号（根据部分对话先后顺序排列）	文件名	音频文件相对路径	音频的文本内容	{0, 1, 2, 3}

label	含义
0	angry
1	happy or excited
2	neutral
3	sad

四.实验结果与分析

结果分析

最终选择了预训练bert+翻译

在该方法中，首先对对话文本进行预处理，添加了说话人的信息和对话主题，以及说话人的说话长度，并且增加了翻译文本，让模型能够从不同语言习惯上找到相似的以及不同的情绪特征，然后使用擅长情感分析的预训练bert模型进行特征提取和模型训练。将focalloss作为损失函数进行训练，通过网格搜索和贝叶斯优化对模型参数进行调优，最终在验证集上评估模型性能。实验结果显示，模型在验证集上达到了较高的准确率和F1分数，表明所提方法能够有效识别四种不同的情感状态。

该方法在调参时使用了贝叶斯优化，针对学习率 `lr`、focalloss的 `alpha` 和 `gamma`、正则化系数 `weight_decay`、预热步数比例 `warmup_fraction` 等参数进行最优参数搜索，调试过程中发现，学习率选择`lr=1e-5~3e-5`之间比较好，正则化强度适中，选择`1e-3~1e-4`，`gamma`选择1比较好，`alpha`选择1.5-2之间更好，情感类别之间的关系不复杂，只有excited和happy合并在一起，需要内部多一层区分。因为类别分布不均衡，所以选择使用focalloss。

1.数据预处理的影响

数据预处理步骤，包括文本清洗、上下文信息添加、情感标签添加和文本增强，显著提高了模型的性能。特别是添加说话人信息和句子长度信息，以及翻译文本的加入，使得模型能够从不同角度捕捉情感特征，提高了模型的泛化能力。

2.模型架构的选择

在模型架构的选择上，我们发现直接将预训练bert模型的嵌入表示送入全连接层的方法，相比于将嵌入表示送入TextCNN的方法，能够获得更好的性能。这可能是因为直接使用预训练bert的嵌入表示能够更好地保留文本的语义信息，而TextCNN可能在特征转换过程中丢失了一些重要信息。

3.超参数调整的效果

通过贝叶斯优化对模型的超参数进行调整，我们找到了最优的参数组合，包括学习率、Focal Loss的alpha和gamma、权重衰减等。这些参数的最优选择，使得模型在验证集上的性能得到了进一步提升。特别是Focal Loss的引入，有效地解决了类别不平衡问题，提高了模型对少数类别的识别能力。

方法优势分析

1. 数据预处理的有效性

上下文信息添加：

- 通过添加说话人信息和句子长度信息，模型能够更好地理解情感表达的上下文，这对于情感识别至关重要。例如，不同性别和情感状态的人在表达方式上可能存在差异，而句子长度可能与情感强度相关。这种方法的优势在于它提高了模型对情感细微差别的识别能力，尤其是在对话数据中，上下文对于理解情感状态非常重要。

情感标签添加：

- 将情感标签与文本结合，使模型在学习过程中可以同时关注文本内容和情感标签，提高了模型对情感类别的识别能力。这种方法的优势在于它利用了数据集中已有的标签信息，增强了模型对情感类别的判别能力。

文本增强：

- 通过翻译、同义词替换等方式增加文本内容，提高了模型的泛化能力，使模型能够学习到更多样的情感表达方式。这种方法的优势在于它通过增加样本多样性来提高模型的鲁棒性，尤其是在处理复杂情感表达时，样本多样性可以帮助模型更好地捕捉情感特征。

2. 模型架构的适应性

预训练模型的选择：

- 微调的预训练模型在预训练阶段使用了更大的数据集，并且采用了动态mask策略和更长的序列长度，这使得它在捕捉语言模式和语义信息方面更为强大。这种方法的优势在于它为情感识别任务提供了更丰富的语言特征，从而提高了模型的性能。

损失函数的选择：

- 使用Focal Loss解决了类别不平衡问题，提高了模型对少数类别的识别能力。这种方法的优势在于它通过调整损失函数来优化模型的训练过程，尤其是在处理不平衡数据集时，能够提高模型的公平性和准确性。

3. 超参数优化的重要性

贝叶斯优化的使用：

- 贝叶斯优化通过构建目标函数的概率模型来指导搜索最优参数，这种方法比传统的网格搜索或随机搜索更高效。这种方法的优势在于它能够利用已有的评估信息来预测未知区域的函数值，从而以更少的迭代次数找到最优解，提高了模型训练的效率 and 效果。

4. 损失函数的创新性

Focal Loss的引入：

- Focal Loss通过增加易分类样本的权重和减少难分类样本的权重，解决了类别不平衡问题，提高了模型对少数类别的识别能力。这种方法的优势在于它通过调整损失函数来优化模型的训练过程，尤其是在处理不平衡数据集时，能够提高模型的公平性和准确性。

综上所述，通过综合应用数据预处理、模型架构选择、超参数优化和损失函数创新等方法，实验在情感识别任务上取得了较好的性能，验证了所提出方法的有效性。这些方法的优势在于它们共同提高了模型对情感状态的识别能力，增强了模型的泛化能力和鲁棒性。

模型解释性

1. 数据集特性对模型解释性的影响

- **特征分布**：在预训练bert模型中，特征分布的不均匀性或极端值可能导致模型在某些样本上过于敏感，从而依赖特定的文本特征进行分类。这种依赖性会降低模型的泛化能力，因为模型可能过度适应训练数据中的特定模式，而不是学习到更广泛的、可推广的情感模式。
- **类别不平衡**：数据集中的类别不平衡问题可能导致模型偏向于多数类，从而影响模型的解释性。特别是在情感识别任务中，某些情感状态的样本可能远多于其他状态。这种不平衡可能导致模型偏向于多数类，从而忽视少数类的特征和模式。通过调整 `class_weight` 参数，可以给予少数类更高的权重，使模型更加关注这些类别，提高对少数类的识别能力，进而增强模型的公平性和解释性。
- **特征相关性**：数据集中特征之间的相关性也会影响模型的解释性。在bert模型中，每个特征都被视为独立的，但实际上，高度相关的特征可能导致模型过度依赖某些特征，忽视其他同样重要的特征。这种依赖性不仅影响模型的性能，也降低了模型的解释性，因为模型的预测可能过于依赖少数几个特征，而不是整个特征集的联合效应。

2. 结合参数和数据集进行模型解释

- **学习率 (lr)**：学习率是控制模型训练过程中权重更新步长的关键参数。通过贝叶斯优化，我们发现学习率在 1×10^{-5} 到 3×10^{-5} 之间时模型表现较好。这表明在训练过程中，较小的学习率有助于模型更细致地逼近最优解，同时避免过大的步长导致训练过程中的震荡或发散。
- **Focal Loss的alpha和gamma**：Focal Loss是一种专为处理类别不平衡问题设计的损失函数。通过贝叶斯优化确定了alpha在1.5到2之间，gamma选择1时，模型对少数类别的识别能力得到提升。这有助于模型在面对情感类别分布不均衡时，更好地关注那些难以分类的样本，从而提高模型的公平性和准确性。
- **正则化系数 (weight_decay)**：正则化是防止模型过拟合的重要手段。通过贝叶斯优化选择了适中的正则化强度，范围在 1×10^{-3} 到 1×10^{-4} 之间。适当的正则化强度有助于模型在保持对训练数据良好拟合的同时，避免对训练数据中的噪声过度敏感，从而提高模型的泛化能力。
- **预热步数比例 (warmup_fraction)**：预热步数是指在训练初期，学习率从较小值逐渐增加到设定值的过程。通过贝叶斯优化确定了预热步数比例，这有助于模型在训练初期更平稳地适应，避免因初始学习率过大而导致的训练不稳定。

通过这些参数的优化，不仅提高了模型的准确性和鲁棒性，还增强了模型的可解释性。这些参数的选择反映了我们在模型性能和解释性之间寻求平衡的努力，使得模型的使用者能够更好地理解和信任模型的决策过程。这种对模型内部工作机制的深入理解，对于提升模型在实际应用中的有效性和可信度至关重要。

五.总结与展望

本实验通过音频单模态和机器学习方法实现了情感识别任务。实验结果表明，所提出的模型在给定数据集上具有良好的性能。但是当前工作仍缺少一些更为细致的考虑：

1. **数据预处理的深化研究：** 未来的研究可以探索更为先进的数据预处理方法，例如进行数据增强，通过大量文本的训练从而适应通用情绪识别，也可以通过增加翻译语种，在现有实现中我补充了法语翻译，确实有效果，可能不同的语种可以分析到更加细微的情绪，比如法语中情感表达更加浪漫和含蓄，可能有助于区分neutral的情绪。还可以从讲话人信息补充出发，继续补充，从而方便bert模型结合其他信息进行分析。也可以在数据生成上深入研究，例如使用Gan联合的优化策略，不断提高评判器和训练器的性能，以进一步提升模型的鲁棒性。
2. **模型架构的创新：** 通过优化现在的模型架构，比如使用最新的mamba模型，或者针对bert的输出特征做进一步的优化，当前的textcnn就有很好的效果，还没有尝试融合到最后的模型上去，加上可能有一点提高，但是仍需要注意过拟合的问题。当然也可以增加合理的对抗样本生成器，通过加入对抗本来提高模型的鲁棒性，从而提高情感识别的准确性。
3. **多模态情感识别的探索：** 未来的研究可以考虑融合音频、视频等多种模态信息，进行多模态情感识别研究，这可能为情感状态的全面理解提供更丰富的信息。
4. **模型解释性的研究：** bert这种基于深度模型的方法解释性比较差，但是仍然可以通过其模型特征进行分析，但未来的研究可以进一步探索模型解释性的方法，如特征重要性分析、局部可解释模型-agnostic解释（LIME）等，以增强模型的可解释性和透明度。
5. **尝试进行编码转换：** 将文本转换为模型可以理解的数值形式，例如使用词嵌入（Word Embeddings）技术，如Word2Vec或GloVe，将文本转换为向量形式，以便模型可以进行数学运算，这样可以尝试更多的模型。

六.参考资料

- [1] [Bert文本分类实践（三）：处理样本不均衡和提升模型鲁棒性trick - 盛小贱吖 - 博客园](#)
- [2] [何恺明大神的「Focal Loss」，如何更好地理解？ - 知乎](#)
- [3] [\[1605.06206\] Equilibrium vortex lattices of a binary rotating atomic Bose-Einstein condensate with unequal atomic masses](#)
- [4] [meghanadh/finetune bert iemocap text · Hugging Face](#)